

1. Le modèle de boîte CSS (margin, border, padding)

Objectif

Comprendre les différentes parties d'une boîte CSS :

- **margin** : espace extérieur
- **border** : bordure
- **padding** : espace intérieur
- **content** : contenu
- **width** : largeur
- **box-sizing** : façon dont la largeur est calculée

Principe : modifier les valeurs du CSS et observer le résultat.

Consignes de création de l'arborescence

1. **Créez un dossier** nommé : **HTML_CSS**
2. **À l'intérieur de ce dossier**, créez les 4 fichiers suivants (fichiers HTML vides pour l'instant) :
 - **01-box-model.html**
 - **02-display.html**
 - **03-flexbox-bas.html**
 - **04-etoiles.html**
3. Votre arborescence finale doit ressembler à ceci :

```
HTML_CSS/  
├── 01-box-model.html  
├── 02-display.html  
├── 03-flexbox-bas.html  
└── 04-etoiles.html
```

4. **Vérifiez** que :
 - Le dossier s'appelle bien **HTML_CSS** (respectez exactement la casse et l'underscore).
 - Les fichiers sont bien **dans** ce dossier et pas ailleurs.
 - Les noms de fichiers sont exactement comme indiqué (chiffres, tirets, extension **.html**).

1.1. Préparation

Copier le code suivant dans le fichier **01-box-model.html**.

```
<!DOCTYPE html>  
<html lang="fr">
```

```
<head>
  <meta charset="UTF-8">
  <title>Le modèle de boîte CSS</title>

  <style>

    body {
      font-family: Arial, sans-serif;
      background-color: #eeeeee;
    }

    .box {
      width: 200px;

      /* À MODIFIER */
      margin: 30px auto;

      /* À MODIFIER */
      border: 10px solid #3498db;

      /* À MODIFIER */
      padding: 20px;

      background-color: #b8f2c8;
    }

    .content {
      background-color: white;
      padding: 15px;
      text-align: center;
    }

    .bandeau {
      background-color: #fff3a6;
      min-height: 50px;
      padding: 15px;
      box-sizing: border-box;
      text-align: center;
    }

  </style>
</head>

<body>

  <h1>Le modèle de boîte CSS</h1>

  <p>
    Modifier les valeurs dans le CSS et observer ce qui change.
  </p>

  <div class="bandeau">
    Élément précédent
  </div>
```

```
<div class="box">
  <div class="content">
    Contenu
  </div>
</div>

<div class="bandeau">
  Élément suivant
</div>

</body>

</html>
```

1.2. Exercice 1 — Modifier le padding

Dans `.box`, repérer :

```
padding: 20px;
```

Teste successivement :

```
padding: 5px;
```

puis :

```
padding: 30px;
```

puis :

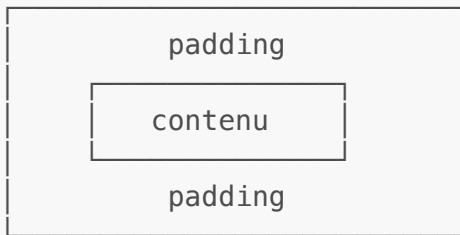
```
padding: 60px;
```

Questions

1. Que devient l'espace entre la bordure bleue et le contenu blanc ?
2. Est-ce que le contenu se déplace ?
3. Est-ce que la bordure se déplace ?
4. Est-ce que la boîte devient plus grande ?

Remarque

Le **padding** crée un espace **entre la bordure et le contenu**.



1.3. Exercice 2 — Modifier la border

Remettre :

```
padding: 20px;
```

Puis modifier :

```
border: 10px solid #3498db;
```

Tester :

```
border: 2px solid #3498db;
```

puis :

```
border: 20px solid #3498db;
```

Puis tester :

```
border: 10px dashed #3498db;
```

Questions

1. Que change la première valeur ?
2. Que change **solid** ou **dashed** ?
3. Que représente **#3498db** ?

Remarque

Cette écriture :

```
border: 10px solid #3498db;
```

correspond à :

```
border-width: 10px;  
border-style: solid;  
border-color: #3498db;
```

1.4. Exercice 3 — Modifier la margin

Remettre :

```
border: 10px solid #3498db;
```

Puis observer :

```
margin: 30px auto;
```

Remplacer par :

```
margin: 10px auto;
```

puis :

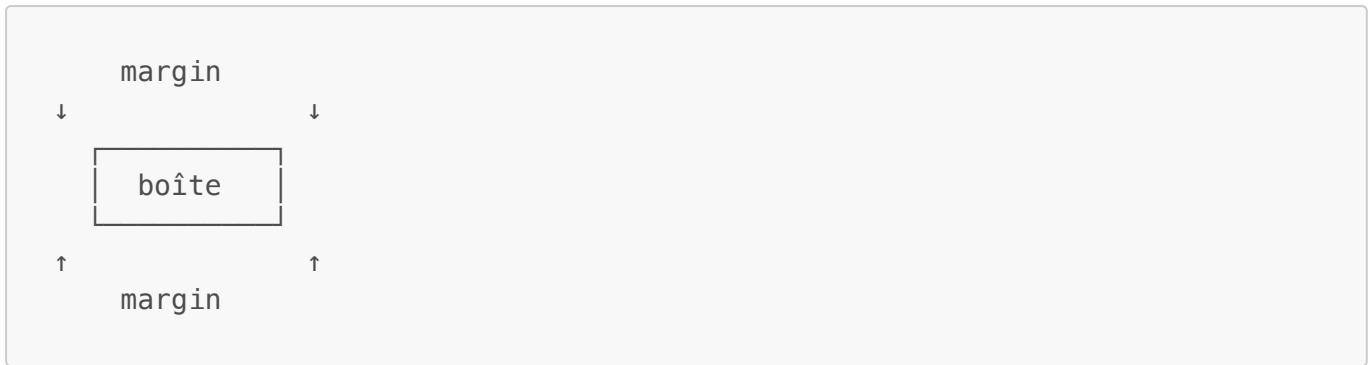
```
margin: 60px auto;
```

Questions

1. Quelle zone est modifiée ?
2. Le contenu change-t-il de taille ?
3. La bordure change-t-elle de taille ?
4. Où se trouve la marge ?

Remarque

La **margin** crée un espace à l'**extérieur de la boîte**.



1.5. Exercice 4 — Comprendre les quatre côtés

Remplacer :

```
margin: 30px auto;
```

par :

```
margin: 10px 20px 30px 40px;
```

Observer le résultat.

Question

À quoi correspondent les quatre valeurs ?

La règle est :

haut → droite → bas → gauche

Donc :

```
margin: 10px 20px 30px 40px;
```

signifie :

```
haut    = 10px  
droite  = 20px  
bas     = 30px  
gauche  = 40px
```

Manipulation

Essayer maintenant :

```
margin-top: 50px;
```

Puis :

```
margin-left: 100px;
```

Puis :

```
margin-bottom: 80px;
```

Observer quelle partie de la page est modifiée.

Remarque

Avec quatre valeurs :

```
margin: haut droite bas gauche;
```

On tourne dans le sens des aiguilles d'une montre :

```
      haut
      ↑
      |
gauche ← boîte → droite
      |
      ↓
      bas
```

Le même principe fonctionne avec **padding**.

1.6. Exercice 5 — Comprendre **margin: auto**

Remettre :

```
margin: 30px auto;
```

Observer que la boîte est centrée horizontalement.

Essayer :

```
margin: 30px;
```

Puis :

```
margin: 30px 50px;
```

Puis :

```
margin: 30px auto;
```

Question

Pourquoi la troisième version centre-t-elle la boîte ?

Remarque

Avec :

```
margin: 30px auto;
```

on obtient :

```
haut      = 30px  
droite    = auto  
bas       = 30px  
gauche    = auto
```

Le navigateur répartit l'espace disponible entre les marges gauche et droite.

1.7. Exercice 6 — Comprendre `width`

Remettre :

```
margin: 30px auto;
```

Dans `.box`, on trouve :

```
width: 200px;
```

Changer la largeur :

```
width: 300px;
```

Puis :

```
width: 400px;
```

Question

La valeur `width` correspond-elle à toute la boîte ?

Pour répondre, regarder le CSS :

```
.box {  
  width: 200px;  
  border: 10px solid #3498db;  
  padding: 20px;  
}
```

Par défaut, `width` correspond à la largeur du **contenu**.

Il faut donc ajouter :

```
contenu  
+ padding gauche  
+ padding droite  
+ border gauche  
+ border droite
```

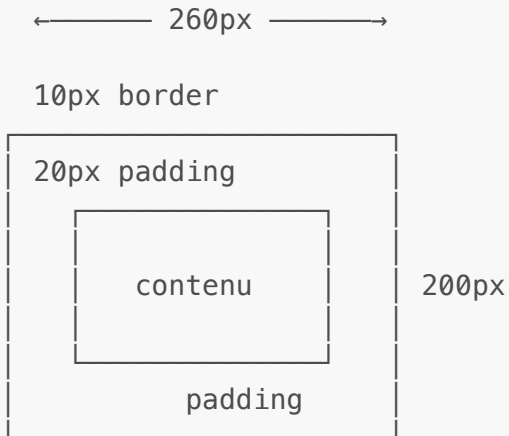
Avec :

```
width = 200px  
padding = 20px  
border = 10px
```

on obtient :

$$200 + 20 + 20 + 10 + 10$$
$$= 260\text{px}$$

La boîte mesure donc **260px de large**.



1.8. Exercice 7 — `box-sizing`

Nous allons maintenant modifier la façon dont le navigateur calcule la largeur.

Ajouter au début du CSS :

```
* {  
  box-sizing: border-box;  
}
```

Le code devient :

```
* {  
  box-sizing: border-box;  
}  
  
body {  
  ...  
}
```

Observer maintenant la boîte.

Elle possède toujours :

```
width: 200px;  
padding: 20px;  
border: 10px solid #3498db;
```

Mais sa largeur totale est maintenant :

200px

et non plus :

260px

Question

Où sont passés les 40px de padding et les 20px de bordure ?

Ils sont maintenant **compris dans les 200px**.

La zone de contenu est donc réduite.

1.9. Comparaison

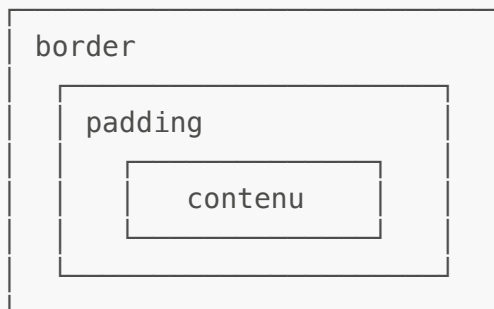
Sans `box-sizing: border-box`

width = 200px

contenu = 200px

padding = 40px

border = 20px

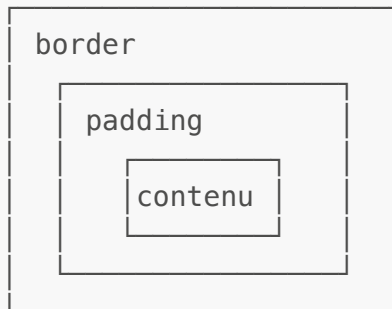


largeur totale = 260px

Avec `box-sizing: border-box`

`width = 200px`

`contenu + padding + border = 200px`



`largeur totale = 200px`

1.10. Exercices

Sans modifier le HTML, réaliser les objectifs suivants uniquement avec le CSS.

Exercice A

Faire une boîte de **300px de largeur totale**.

Utiliser :

```
box-sizing: border-box;
```

Exercice B

Faire une bordure de :

- 5px
- rouge
- en pointillés

Exercice C

Ajouter :

20px en haut
40px à droite
30px en bas
10px à gauche

avec une seule propriété **padding**.

Exercice D

Centrer horizontalement la boîte.

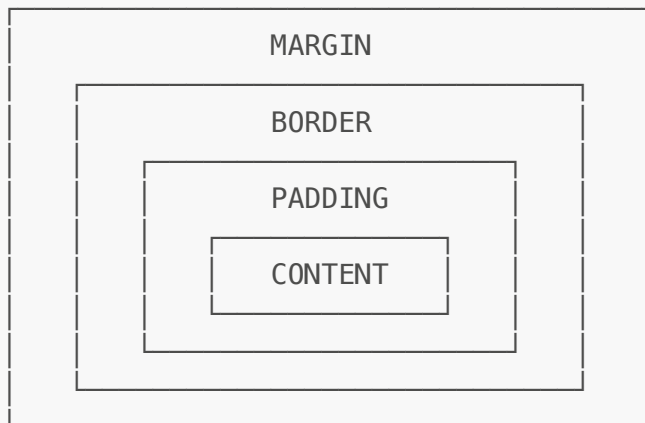
Exercice E

Faire en sorte que le contenu soit très éloigné de la bordure.

Quel paramètre doit-on modifier ?

1.11. Synthèse

Le modèle de boîte peut être représenté ainsi :



Propriété	Rôle
margin	espace extérieur
border	bordure
padding	espace intérieur
content	contenu
width	largeur de référence
box-sizing	façon de calculer width et height

La règle essentielle

```
MARGIN
↓
BORDER
↓
PADDING
↓
CONTENT
```

Conclusion

```
margin → dehors
border → bordure
padding → dedans
content → contenu
```

Et :

```
box-sizing: border-box;
```

permet de dire :

```
« La largeur qu'on donne comprend déjà le padding et la bordure. »
```

2. `display` : block, inline et inline-block

Objectif

Comprendre comment les éléments se placent les uns par rapport aux autres.

Nous allons comparer :

```
display: block;
display: inline;
display: inline-block;
```

A. Manipulation

Copier le code ci-dessous dans le fichier `02-display.html`

```
<!DOCTYPE html>
<html lang="fr">

<head>
<meta charset="UTF-8">
<title>Display</title>

<style>

    body {
        font-family: Arial, sans-serif;
        background-color: #eeeeee;
    }

    .bloc {
        /* À MODIFIER */
        display: block;

        background-color: #b8f2c8;
        border: 3px solid #249447;

        padding: 20px;
        margin: 10px;
    }

</style>
</head>

<body>

<h1>La propriété display</h1>

<p>
    Modifie <strong>display</strong> et observe ce qui change.
</p>

<div class="bloc">Élément 1</div>
<div class="bloc">Élément 2</div>
<div class="bloc">Élément 3</div>

</body>

</html>
```

B. Première manipulation : `display: block`

Commencer avec :

```
display: block;
```

Observer les trois éléments.

Ils sont placés les uns sous les autres :

Élément 1

Élément 2

Élément 3

Un élément **block** commence sur une nouvelle ligne.

Par défaut, un élément **block** occupe généralement toute la largeur disponible.

C. Deuxième manipulation : **display: inline**

Remplacer :

```
display: block;
```

par :

```
display: inline;
```

Observer le résultat.

Élément 1

Élément 2

Élément 3

Les éléments restent maintenant **dans la même ligne**, lorsque l'espace disponible le permet.

Question

Que s'est-il passé ?

À retenir

Un élément `inline` se comporte comme une partie du texte.

Exemple :

```
<p>
  Bonjour
  <span>tout le monde</span>
</p>
```

Le `span` reste dans la ligne.

D. Une observation importante

Avec :

```
display: inline;
```

essayer d'ajouter :

```
width: 200px;
height: 100px;
```

par exemple :

```
.bloc {
  display: inline;

  width: 200px;
  height: 100px;

  background-color: #b8f2c8;
  border: 3px solid #249447;
}
```

Observer ce qui se passe.

La largeur et la hauteur ne se comportent pas comme avec un élément `block`.

E. Troisième manipulation : `inline-block`

Remplacer :

```
display: inline;
```

par :

```
display: inline-block;
```

Puis ajouter :

```
width: 150px;
```

Les éléments peuvent maintenant rester sur la même ligne tout en ayant une largeur définie.

Élément 1

Élément 2

Élément 3

Remarque

inline-block combiner certaines caractéristiques des deux.

block
→ nouvelle ligne

inline
→ reste dans la ligne

inline-block
→ reste dans la ligne
→ mais permet notamment de définir une largeur et une hauteur

F. Exercice

Sans modifier le HTML :

```
<div class="bloc">Élément 1</div>  
<div class="bloc">Élément 2</div>  
<div class="bloc">Élément 3</div>
```

on veut obtenir :



avec des éléments de 150px de largeur.

Solution attendue

```
display: inline-block;
width: 150px;
```

Conclusion

display	Comportement
block	commence sur une nouvelle ligne
inline	reste dans la ligne
inline-block	reste dans la ligne et accepte notamment width et height

3. Flexbox : placer un élément en bas

Objectif

Nous allons utiliser Flexbox pour placer un bouton **tout en bas d'une carte**.

A. La carte

Copier le code ci-dessous dans le fichier 03-flexbox-bas.html

```
<!DOCTYPE html>
<html lang="fr">

<head>
<meta charset="UTF-8">
<title>Flexbox</title>

<style>

    body {
        font-family: Arial, sans-serif;
```

```
        background-color: #eeeeee;
    }

    .card {
        width: 300px;
        height: 400px;

        background-color: white;
        border: 3px solid #3498db;
        padding: 20px;

        /* À MODIFIER */
    }

    .button {
        background-color: #3498db;
        color: white;

        padding: 15px;
        text-align: center;

        /* À MODIFIER */
    }

</style>
</head>

<body>

<h1>Flexbox</h1>

<div class="card">

    <div class="content">

        <h2>La carte</h2>

        <p>
            Voici du contenu dans la carte.
        </p>

        <p>
            Le bouton doit être placé tout en bas.
        </p>

    </div>

    <div class="button">
        Bouton en bas
    </div>

</div>

</body>
```

```
</html>
```

B. Étape 1 — `display: flex`

Ajouter :

```
.card {  
  display: flex;  
}
```

Observer.

Les deux enfants de `.card` (`.content` et `.button`) sont maintenant disposés selon le fonctionnement de Flexbox.

Question

Qu'est-ce qui change par rapport à une disposition normale ?

C. Étape 2 — `flex-direction: column`

Ajouter :

```
.card {  
  display: flex;  
  flex-direction: column;  
}
```

Les éléments sont maintenant placés verticalement :



contenu

bouton

Remarque

```
display: flex;
```

active Flexbox.

```
flex-direction: column;
```

place les éléments suivant un axe vertical.

D. Étape 3 — pousser le bouton vers le bas

Nous voulons maintenant obtenir :



Ajouter :

```
.button {  
  margin-top: auto;  
}
```

Le bouton descend automatiquement.

E. Que fait `margin-top: auto` ?

La marge supérieure prend tout l'espace disponible.

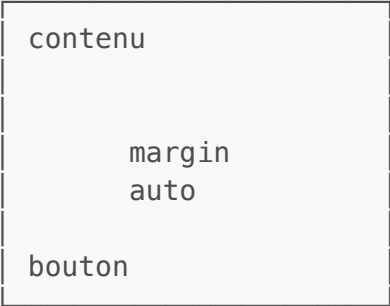




bouton

On peut l'imaginer ainsi :

margin-top: auto



contenu

margin
auto

bouton

F. Manipulations

Manipulation 1

Supprimer :

```
display: flex;
```

Que devient le bouton ?

Manipulation 2

Remettre :

```
display: flex;
```

mais supprimer :

```
flex-direction: column;
```

Que se passe-t-il ?

Manipulation 3

Remettre :

```
flex-direction: column;
```

mais supprimer :

```
margin-top: auto;
```

Que se passe-t-il ?

Manipulation 4

Modifier la hauteur :

```
height: 400px;
```

Essayer :

```
height: 300px;
```

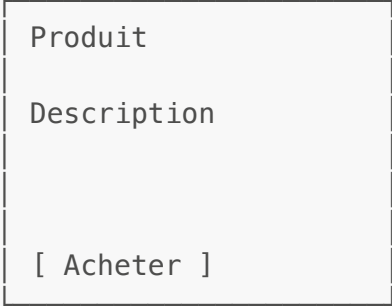
puis :

```
height: 500px;
```

Que devient l'espace entre le contenu et le bouton ?

G. Exercices

Transformer cette carte :



Produit

Description

[Acheter]

pour que le bouton reste en bas, même si la description change.

Les trois lignes importantes

```
.card {  
  display: flex;  
  flex-direction: column;  
}  
  
.button {  
  margin-top: auto;  
}
```

Conclusion

```
display: flex;
```

→ active Flexbox.

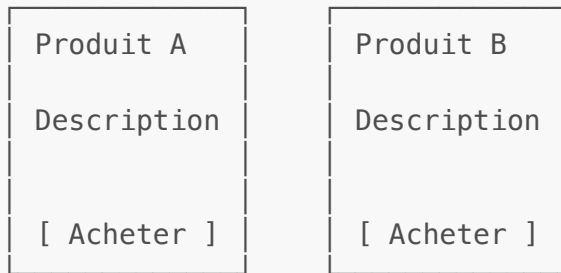
```
flex-direction: column;
```

→ organise les éléments verticalement.

```
margin-top: auto;
```

→ utilise l'espace disponible pour pousser le bouton vers le bas.

C'est une technique pratique pour avoir, par exemple, plusieurs cartes de produits avec un bouton toujours aligné en bas :



4. Créer une notation par étoiles

Objectif

Nous allons utiliser :

- `display: flex`
- les pseudo-éléments `::before` et `::after`
- `position: relative`
- `position: absolute`
- `width`
- `overflow: hidden`

pour créer une notation par étoiles.

A. Le code de départ

Copier le code ci-dessous dans le fichier : `04-etoiles.html`

```
<!DOCTYPE html>
<html lang="fr">

<head>
<meta charset="UTF-8">
<title>Notation par étoiles</title>

<style>

    .rating {
        display: flex;
        gap: 5px;
    }

    .star {
        position: relative;

        width: 40px;
```

```
        height: 40px;

        font-size: 40px;
        line-height: 40px;
    }

    /* Étoile grise */
    .star::before {
        content: "★";

        color: #ccc;

        position: absolute;
        left: 0;
        top: 0;
    }

    /* Étoile entièrement jaune */
    .full::after {
        content: "★";

        color: #ffc107;

        position: absolute;
        left: 0;
        top: 0;
    }

    /* Demi-étoile jaune */
    .half::after {
        content: "★";

        color: #ffc107;

        position: absolute;
        left: 0;
        top: 0;

        width: 50%;
        overflow: hidden;
    }

</style>
</head>

<body>

<h2>Note : 1,5 / 3</h2>

<div class="rating">

    <span class="star full"></span>

    <span class="star half"></span>
```

```
<span class="star"></span>

</div>

</body>

</html>
```

Le résultat doit être :

★★☆

avec :

- une étoile jaune ;
- une demi-étoile jaune ;
- une étoile grise.

B. Première manipulation : `display: flex`

Observer :

```
.rating {
  display: flex;
  gap: 5px;
}
```

Supprimer temporairement :

```
display: flex;
```

Que se passe-t-il ?

Remettre ensuite :

```
display: flex;
```

Remarque

Flexbox permet de placer facilement les trois étoiles côte à côte.

★ ★ ★

C. Deuxième manipulation : l'étoile grise

Observer :

```
.star::before {  
  content: "★";  
  color: #ccc;  
}
```

Le pseudo-élément `::before` crée une étoile grise.

Chaque élément :

```
<span class="star"></span>
```

possède donc une étoile grise.

Essayer de modifier :

```
color: #ccc;
```

par :

```
color: red;
```

Puis :

```
color: blue;
```

Question

Pourquoi toutes les étoiles changent-elles de couleur ?

D. Troisième manipulation : l'étoile pleine

Observer :

```
.full::after {  
  content: "★";  
  color: #ffc107;  
}
```

Le sélecteur :

```
.full
```

ne concerne que :

```
<span class="star full"></span>
```

Cette étoile possède donc :

- une étoile grise créée par `::before` ;
- une étoile jaune créée par `::after`.

L'étoile jaune est placée exactement par-dessus l'étoile grise.

Le jaune cache donc complètement le gris.

E. Quatrième manipulation : la demi-étoile

Observe :

```
.half::after {  
  content: "★";  
  color: #ffc107;  
  
  position: absolute;  
  left: 0;  
  top: 0;  
  
  width: 50%;  
  overflow: hidden;  
}
```

La propriété importante est :

```
width: 50%;
```

Elle limite la largeur de la zone contenant l'étoile jaune.

Et :

```
overflow: hidden;
```

cache la partie de l'étoile qui dépasse de cette zone.

On obtient donc la moitié droite de l'étoile jaune est cachée.

F. Manipulation : modifier la note

Changer :

```
width: 50%;
```

par :

```
width: 25%;
```

Puis :

```
width: 75%;
```

Puis :

```
width: 100%;
```

Observer le résultat.

Question

Que se passe-t-il avec :

```
width: 100%;
```

On devrait obtenir une étoile entièrement jaune.

G. Comprendre position

La règle :

```
.star {  
    position: relative;  
}
```

sert de référence pour les éléments positionnés en **absolute**.

Les pseudo-éléments utilisent :

```
position: absolute;  
left: 0;  
top: 0;
```

Ils sont donc placés dans le coin supérieur gauche de **.star**.

On peut schématiser :

`.star`



↑
`left: 0`
`top: 0`

Exercice 1

Modifier le HTML pour obtenir :

★★★★☆

avec une note sur 5.

On doit avoir :

```
<span class="star full"></span>  
<span class="star full"></span>  
<span class="star full"></span>
```

```
<span class="star"></span>
<span class="star"></span>
```

Exercice 2

Obtenir :



On doit utiliser :

- deux étoiles `.full` ;
- une étoile `.half` ;
- deux étoiles `.star`.

Conclusion

Pour créer une étoile vide :

```
.star::before {
  content: "★";
  color: #ccc;
}
```

Pour créer une étoile pleine :

```
.full::after {
  content: "★";
  color: #ffc107;
}
```

Pour créer une demi-étoile :

```
.half::after {
  content: "★";
  color: #ffc107;
  width: 50%;
  overflow: hidden;
}
```

La combinaison :

```
position: absolute;
width: 50%;
overflow: hidden;
```

permet de ne conserver que la moitié de l'étoile jaune.

Remarque : la variante utilisée dans le projet « boutique »

Dans `style.css` du projet boutique, la même idée sera appliquée avec une légère variante :

- au lieu du caractère ★, on utilise l'icône Font Awesome (`content: "\f005"` avec `font-family: "Font Awesome 7 Free"`);
- au lieu d'un `::after` séparé pour l'étoile pleine, on **recolore directement** l'étoile grise :

```
.stars > label::before {
  font-family: "Font Awesome 7 Free";
  font-weight: 900;
  content: "\f005";
  color: #c9c9c9; /* étoile grise */
}

.stars > .full::before {
  color: #fccf47; /* on recolore la même étoile en jaune */
}
```
